



ರೈ ತಾಂತ್ರಿಕ ವಿಶ್ವವಿದ್ಯಾಲಯ
Rai Technology University

(Estb. under Karnataka Act, No. 40 of 2013)

ASSIGNMENT -1

Subject Title: DEEP LEARNING

Subject Code: OCS351T

Submitted By:

Student Name: MANISH M

University Enrolment Number: RTU24101CS021

Program Name: B.TECH CSE (AIML)

Semester: 5TH Sem

Session: 2026-2027

Submitted To:

Faculty Name: Mr. Satyam sir

Department of Computer Science & Engineering College

Of Engineering

Rai Technology University

REPORT

Aim: TO build a simple neural network using TensorFlow/Keras to classify handwritten digits from 0 to 9 using the MNIST Dataset

objective

- Load and Explore the MNIST Dataset
- Display sample handwritten digit images
- Build and train a neural network
- Evaluate the model using test accuracy
- Visualize training
- Predict 5 handwritten images
- Perform one experiment by changing the neural network architecture

PROCEDURE

- Import Libraries
- Load MNIST Dataset
- Display Sample Images
- Data Preprocessing
- Build Neural Network
- compile the model
- Train the Model
- Evaluate the Model
 - Validation Accuracy

• Validation Loss

- Test the model on 5 Handwritten Images
- Add Dropout
- compile EXPERIMENTAL model
- Train EXPERIMENTAL model
- Evaluate EXPERIMENTAL model
- Compare Results

Conclusion

The experiment demonstrates that a simple neural network can effectively recognize handwritten digits using the MNIST dataset. TensorFlow/Keras provides an easy way to build, train and evaluate deep learning models.

CODE :-

```

import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

(x_train, y_train), (x_test, y_test) = tf.keras.load_data()
Print ("Training images:", x_train.shape)
Print ("Testing images:", x_test.shape)

plt.figure(figsize=(8,4))
for i in range(6):
    plt.subplot(2,3,i,1)
    plt.imshow(x_train[i*20], cmap='gray')
    plt.title("Label: " + str(y_train[i*20]))
    plt.axis('off')
plt.tight_layout()
plt.show()

x_train = x_train('float32')/255.0
x_test = x_test.astype('float32')/255.0
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=(28,28)),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')
])
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

```

```
history = model.fit (
```

```
    x_train,
```

```
    y_train,
```

```
    epochs = 5,
```

```
    batch_size = 64
```

```
    validation_split = 0.2
```

```
)
```

```
print ("In ===== MODEL RESULTS =====")
```

```
print ("Test Loss:", test_loss)
```

```
print ("Test Accuracy:", test_accuracy)
```

```
print ("Test Accuracy Percentage:", test_accuracy*100)
```

```
plt.figure(figsize=(8,5))
```

```
plt.plot (
```

```
    history.history['accuracy'],
```

```
    label = 'Training Accuracy'
```

```
)
```

```
plt.plot (
```

```
    history.history['val-accuracy'],
```

```
    label = 'Validation Accuracy'
```

```
)
```

```
plt.xlabel ('Epoch')
```

```
plt.ylabel ('Accuracy')
```

```
plt.title ('Training and Validation Accuracy')
```

```
plt.legend ()
```

```
plt.grid ()
```

```
plt.show ()
```

```
plt.figure(figsize=(8,5))
```

```
plt.plot (
```

```
    history.history['loss']
```

```

label = 'Training Loss'
plt.plot(
    history.history['val-loss'],
    label = 'Validation Loss'
)
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training and Validation Loss - set 2')
plt.legend()
plt.grid()
plt.show()

plt.figure(figsize=(10, 4))
for i in range(5):
    plt.subplot(1, 5, i, 1)
    plt.imshow(
        x-test[i+10],
        cmap='gray'
    )
    plt.title(
        "Actual:" + str(y-test[i+10]) +
        "\n Predicted:" + str(predicted_labels[i])
    )
    plt.axis('off')
plt.tight_layout()
plt.show()

Predictions = model.predict(x-test[10:15])
predicted_labels = np.argmax(Predictions,
                             axis = 1)

```

```
Print ("\n ===== FIVE IMAGE PREDICTIONS =====")
```

```
for i in range(5):
```

```
    Print (
```

```
        "Image", i+1
```

```
        "Actual =", y_test[i+10],
```

```
        "Predicted =", Predicted_labels[i])
```

```
plt.figure(figsize=(8,5))
```

```
plt.plot (
```

```
    experiment_history.history['loss'],
    label = 'Training Loss'
```

```
plt.plot (
```

```
    experiment_history.history['val_loss'],
    label = 'Validation Loss'
```

```
    plt.xlabel('Epoch')
```

```
    plt.ylabel('Loss')
```

```
plt.title('Experimental Model Loss')
```

```
plt.legend()
```

```
plt.grid()
```

```
plt.show()
```

```
plt.figure(figsize=(8,5))
```

```
plt.plot (
```

```
    experiment_history.history['accuracy'],
    label = 'Training Accuracy'
```

```
plt.plot (
```

```
    experiment_history.history['val-accuracy'],
    label = 'Validation Accuracy'
```

```
plt.xlabel('Epoch')
```

```

plt.ylabel('Accuracy')
plt.title('EXPERIMENTAL Model Accuracy')
plt.legend()
plt.grid()
plt.show()

```

```

Print("\n===== FINAL COMPARISON=====")

```

```

Print (
    "original Model Accuracy:",
    test-accuracy*100, "%"
)

```

```

Print (
    "EXPERIMENTAL Model Accuracy:",
    exp-accuracy*100, "%"
)

```

```

Print("\n===== EXPERIMENTAL MODEL RESULT=====")

```

```

Print ("EXPERIMENTAL Test Loss:", EXP-loss)
      exp-accuracy

```

```

Print ("EXPERIMENTAL Model Accuracy Percentage:",
      exp-accuracy*100, "%")

```

```

experiment_model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dropout(0.3),
    tf.keras.layers.Dense(10, activation='softmax')
])

```

```
Experiment_model.compile(  
    optimizer='adam',  
    loss='sparse_categorical_crossentropy',  
    metrics=['accuracy']  
)
```

```
Experiment_history = Experiment_model.fit(  
    x_train,  
    y_train,  
    epochs=5,  
    batch_size=64,  
    validation_split=0.2  
)
```

```
plt.figure(figsize=(10,4))
```

```
for i in range(5):
```

```
    plt.subplot(1,5,i+1)
```

```
    plt.imshow(  
        x_test[i+10],  
        cmap='gray'  
    ),
```

```
    plt.title(  
        "Actual : " + str(y_test[i+10]) +  
        "In Predicted : " + str(predicted_labels[i])  
    ),
```

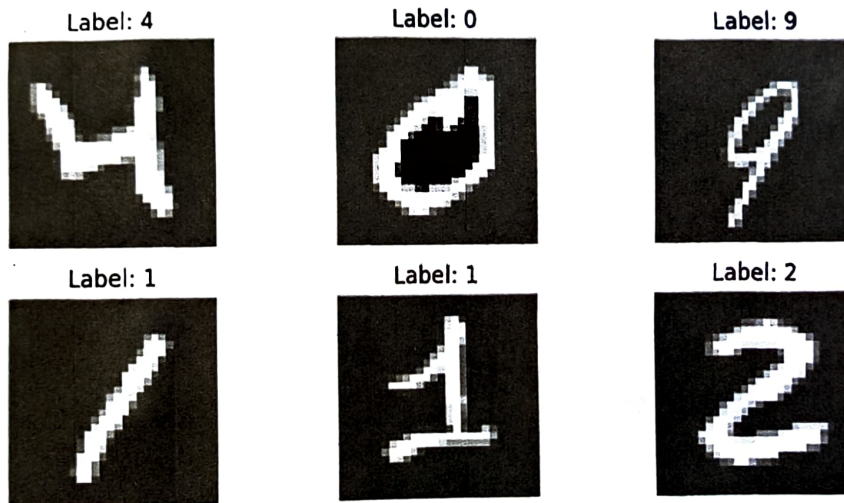
```
    plt.axis('off')
```

```
plt.tight_layout()
```

```
plt.show()
```

OUTPUT'S :-

Training images: (60000, 28, 28)
Testing images: (10000, 28, 28)



Epoch 1/5
750/750 ————— 5s 5ms/step - accuracy: 0.8919 - loss: 0.3878 - val_accuracy: 0.9424 - val_loss: 0.2082
Epoch 2/5
750/750 ————— 5s 6ms/step - accuracy: 0.9469 - loss: 0.1849 - val_accuracy: 0.9548 - val_loss: 0.1615
Epoch 3/5
750/750 ————— 3s 3ms/step - accuracy: 0.9583 - loss: 0.1392 - val_accuracy: 0.9603 - val_loss: 0.1422
Epoch 4/5
750/750 ————— 5s 6ms/step - accuracy: 0.9671 - loss: 0.1103 - val_accuracy: 0.9656 - val_loss: 0.1173
Epoch 5/5
750/750 ————— 4s 5ms/step - accuracy: 0.9732 - loss: 0.0900 - val_accuracy: 0.9660 - val_loss: 0.1104

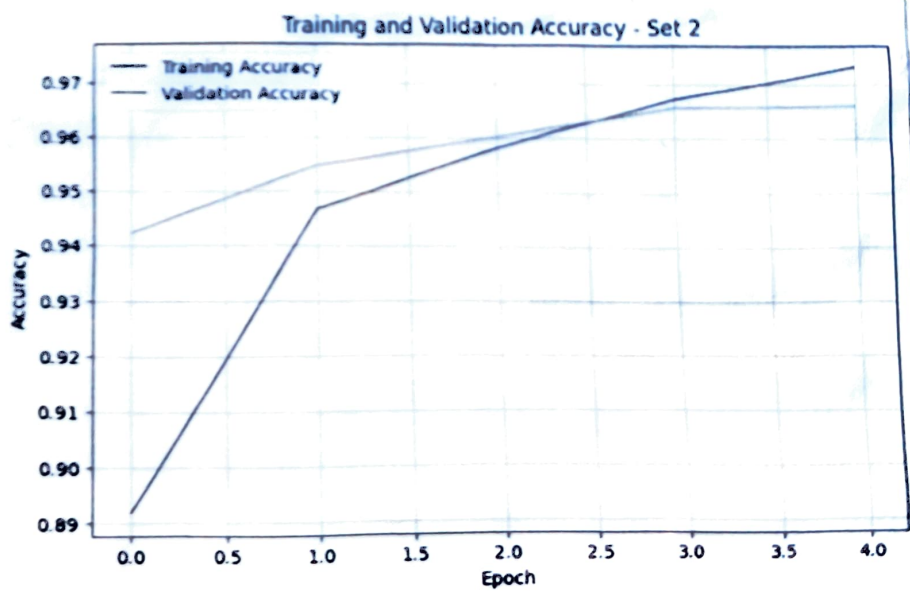
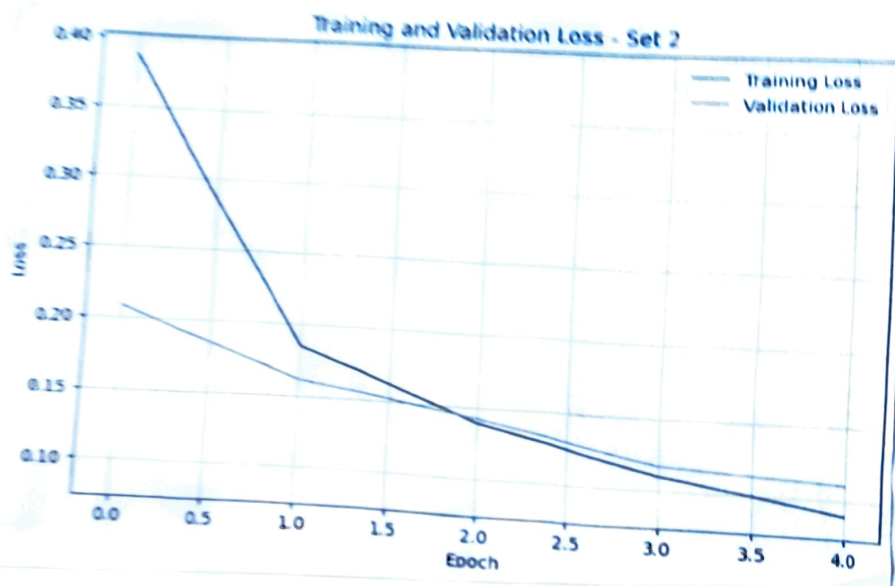
313/313 ————— 1s 2ms/step - accuracy: 0.9687 - loss: 0.1064

----- MODEL RESULTS -----

Test Loss: 0.10642706602811813

Test Accuracy: 0.9686999917030334

Test Accuracy Percentage: 96.86999917030334 %

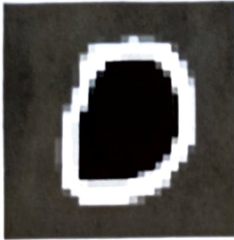


1/1 ————— 0s 142ms/step

----- FIVE IMAGE PREDICTIONS -----

Image 1	Actual = 0	Predicted = 0
Image 2	Actual = 6	Predicted = 6
Image 3	Actual = 9	Predicted = 9
Image 4	Actual = 0	Predicted = 0
Image 5	Actual = 1	Predicted = 1

Actual: 0
Predicted: 0



Actual: 6
Predicted: 6



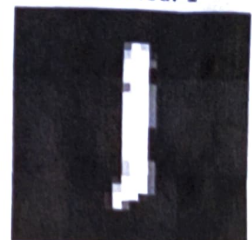
Actual: 9
Predicted: 9



Actual: 0
Predicted: 0



Actual: 1
Predicted: 1



Epoch 1/5

750/750 ————— 4s 4ms/step - accuracy: 0.8572 - loss: 0.4933 - val_accuracy: 0.9360 - val_loss: 0.2233

Epoch 2/5

750/750 ————— 3s 4ms/step - accuracy: 0.9215 - loss: 0.2699 - val_accuracy: 0.9475 - val_loss: 0.1817

Epoch 3/5

750/750 ————— 3s 3ms/step - accuracy: 0.9343 - loss: 0.2249 - val_accuracy: 0.9585 - val_loss: 0.1507

Epoch 4/5

750/750 ————— 2s 3ms/step - accuracy: 0.9442 - loss: 0.1927 - val_accuracy: 0.9606 - val_loss: 0.1358

Epoch 5/5

750/750 ————— 5s 6ms/step - accuracy: 0.9461 - loss: 0.1767 - val_accuracy: 0.9628 - val_loss: 0.1266

313/313 ————— 1s 2ms/step - accuracy: 0.9626 - loss: 0.1247

----- EXPERIMENTAL MODEL RESULTS -----

Experimental Test Loss: 0.12469293177127838

Experimental Test Accuracy: 0.9625999927520752

Experimental Accuracy Percentage: 96.25999927520752 %

===== FINAL COMPARISON =====

Original Model Accuracy: 96.86999917030334 %

Experimental Model Accuracy: 96.25999927520752 %

